

1. (10 puntos) ¿Qué estrategia se utilizó para modelar cada entidad de Parranderos dentro de la base de datos documental? ¿Cuáles relaciones se modelaron como objetos embebidos y cuáles se modelaron como referencias?

Para el modelado de Parranderos se utilizó una estrategia híbrida, combinando el uso de referencias y documentos embebidos según vimos necesario.

Las entidades principales del sistema, como Bares y Bebedores, ya existen en Oracle y poseen una estructura estable y reutilizable. Por esta razón, en MongoDB únicamente se almacenan referencias hacia dichas entidades mediante identificadores (bar_id, bebedor_id). Esta decisión evita la duplicación de información y facilita mantener consistencia entre Oracle y MongoDB.

Por otro lado, las nuevas funcionalidades sociales del sistema —comentarios y eventos— requieren una estructura flexible y dinámica. Debido a esto, se decidió almacenar estas entidades directamente en MongoDB.

Modelado de Eventos: Dentro de Eventos se modeló como referencia: Eventos → Bares, mientras que modelamos como objetos embebidos artistas invitados y promociones

Modelado de Comentarios: Dentro de Comentarios se modelaron como referencias: Comentarios → Bares, Comentarios → Bebedores. Mientras que modelamos como objetos embebidos reacciones y respuestas

2. (10 puntos) ¿Qué estrategia se utilizó para modelar cada entidad dentro de la base de datos documental? ¿Cuáles relaciones se modelaron como objetos embebidos y cuáles se modelaron como referencias?

La estrategia consistió en:referenciar entidades grandes y reutilizables, embeber información pequeña y dependiente, y aprovechar la flexibilidad documental de MongoDB para manejar estructuras variables.

Gracias a esto, el sistema puede adaptarse fácilmente a nuevos requerimientos sin necesidad de modificar esquemas rígidos, permitiendo agregar nuevos atributos en comentarios y eventos de manera sencilla.

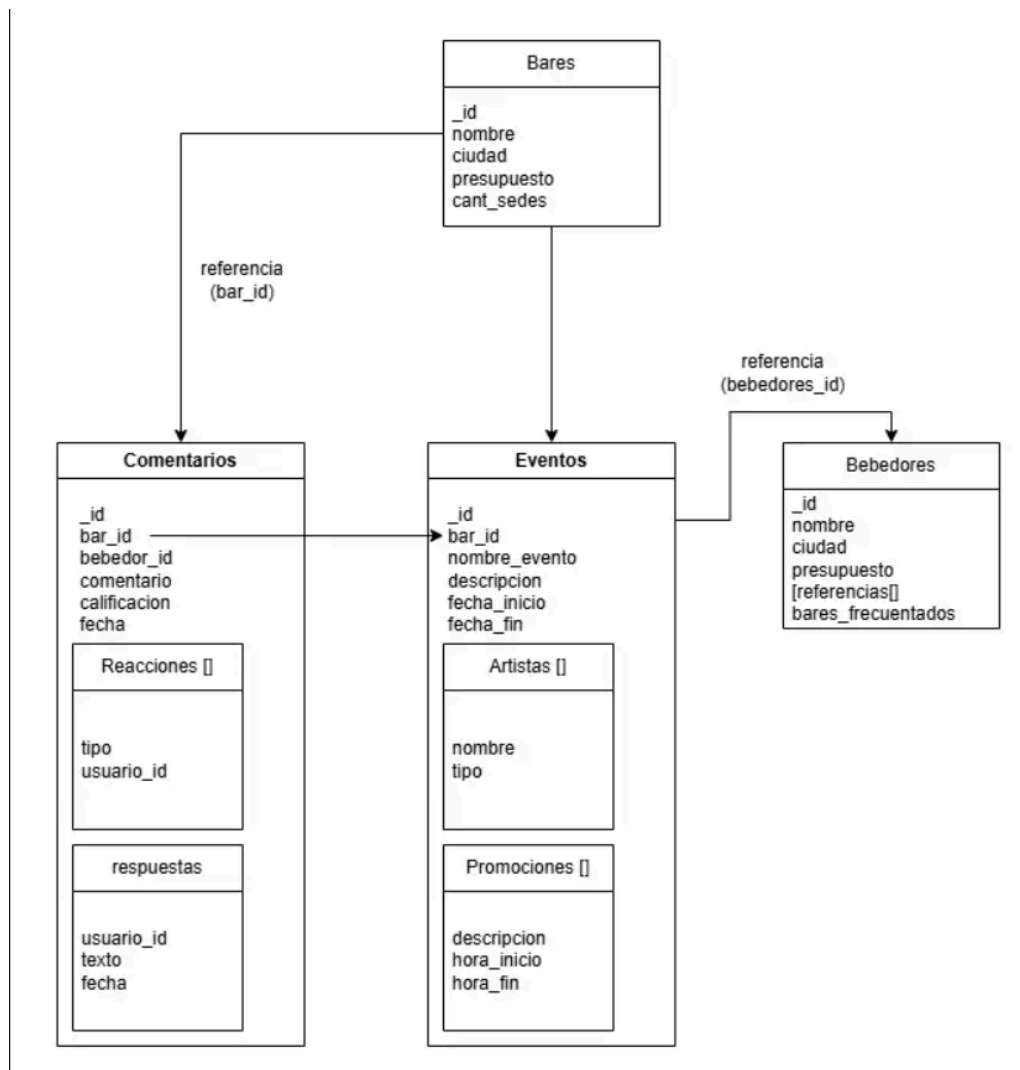
Modelamos como referencia las entidades Bares y bebedores ya que de por si ya están en Oracle y porque no pueden permitir duplicaciones, mientras que los documentos embebidos se utilizaron en estructuras cuya existencia depende completamente del documento principal. Por ejemplo: las respuestas pertenecen únicamente a un comentario, las promociones pertenecen únicamente a un evento.

ACLARACION CODIGO DE ORACLE:

Todo el JavaScript de la pagina de detalle del bar usa fetch() para hacer peticiones a la API sin recargar la pagina, siguiendo un patron estandar que aprendi de MDN Web Docs (developer.mozilla.org), la referencia oficial de las APIs del navegador. El codigo valida que los campos obligatorios no esten vacios, arma el objeto JSON de forma condicional agregando solo los campos opcionales que el usuario completo, limpia los formularios despues de cada envio y recarga las listas automaticamente.

URL EN RENDER: <https://s9-g4-2026-1.onrender.com>

UML OBTENIDO:



APLICACION DE APEX

Detalle Bar

Informacion Bar

Nombre ↑±	Ciudad	Presupuesto	Cant Sedes
Café Tropical	Pereira	Medio	9

1 - 1

Comentarios

Cargando comentarios...

Eventos

Cargando eventos...

Comentario

hola

Nombre del evento

a

Artista

Quevedo

Cover

15000

Enviar Comentario

Registrar Evento

Bares

Order By Nombre	
BARES Latino Santa Marta	BARES Latino Villavicencio
BARES Latino Cali	BARES Latino Pereira
BARES Latino Ibague	BARES Latino BAREStranquilla
BARES Latino BAREStranquilla	BARES Latino Armenia
BARES Latino Manizales	BARES Secreto Bucaramanga

EVIDENCIAS BARES

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/bares/58/comentarios' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "bebedor_id": 289,
    "comentario": "buen ambiente",
    "calificacion": 5
  }'
```

Request URL

http://127.0.0.1:8000/bares/58/comentarios

Server response

Code	Details
200	Response body <pre>{ "mensaje": "Comentario guardado" }</pre> Download Response headers <pre>access-control-allow-origin: * content-length: 33 content-type: application/json date: Sun, 17 May 2026 04:16:56 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/

Server response

Code	Details
200	Response body <pre>{ "estado": "API funcionando correctamente" }</pre> Download Response headers <pre>content-length: 42 content-type: application/json date: Sun, 17 May 2026 04:14:12 GMT server: uvicorn</pre>